

Emerging Topics in Generative AI

**Agent Protocols, Frameworks, and
the New Agentic Web**

Prof. Jal Irani

Lecture Purpose

- Cutting edge technologies adapted by industry professionals today
 - The tools were gathered from these professionals that internally, they are moving to either learn these skills or implement in some capacity
- Adding skills that are desirable to a potential employer
 - Employers hire problem solvers!



Agenda

Prerequisite: Familiarity with MCP (Model Context Protocol) from prior lecture

- 1. A2UI:** Agent-to-User Interface Specification (Google)
- 2. NANDA:** Networked Agents & Decentralized AI (MIT Media Lab)
- 3. AgentKit:** Build, Deploy, Optimize Agents (OpenAI)
- 4. Browser Use:** Visual Agent Interaction Paradigm
- 5. AGENTS.md:** Repo-Level Agent Guidance (OpenAI / AAIL)



The Agentic Protocol Stack

NANDA

Discovery — Find agents globally by capability

A2A

Agent ↔ Agent — Cross-framework communication

MCP

Agent ↔ Tools & Data — Universal tool connection

A2UI

Agent ↔ User — Render interfaces & stream interactions

Infrastructure

AgentKit • NemoClaw • Browser Use • AGENTS.md



Market Context: Why This Matters Now

40%

of enterprise apps will embed AI agents by end of 2026 (Gartner, up from <5% in 2025)

\$52.6B

projected agentic AI market by 2030 at 46.3% CAGR (from \$7.6B in 2025)

150+

organizations supporting A2A protocol (Salesforce, SAP, Microsoft, Atlassian)

97M+

monthly MCP SDK downloads across Python & TypeScript (Dec 2025)

The Agentic AI Foundation (AAIF) which was formed in December 2025 under the Linux Foundation by Anthropic, Block, and OpenAI. Platinum members: AWS, Google, Microsoft, Cloudflare, Bloomberg. Founding projects: MCP, AGENTS.md, goose.



Recap: MCP (Model Context Protocol)

Remember: MCP is the foundational layer of the agentic protocol stack

- **What:** Universal open standard for connecting AI models to external tools, data, and APIs
- **Created:** Anthropic, November 2024. Donated to Linux Foundation AAF, December 2025
- **Adoption:** 100M+ monthly SDK downloads. 100+ connectors in Claude. OpenAI + Google support
- **Architecture:** Client-Server model. Servers expose tools/resources. Clients (LLM apps) consume them

MCP 2026 Roadmap Priorities:

- **Transport scalability:** Evolve Streamable HTTP for horizontal scaling and load balancers
- **Security:** DPoP, Workload Identity Federation, vulnerability disclosure program
- **Enterprise readiness:** SSO integration, gateway patterns, configuration portability



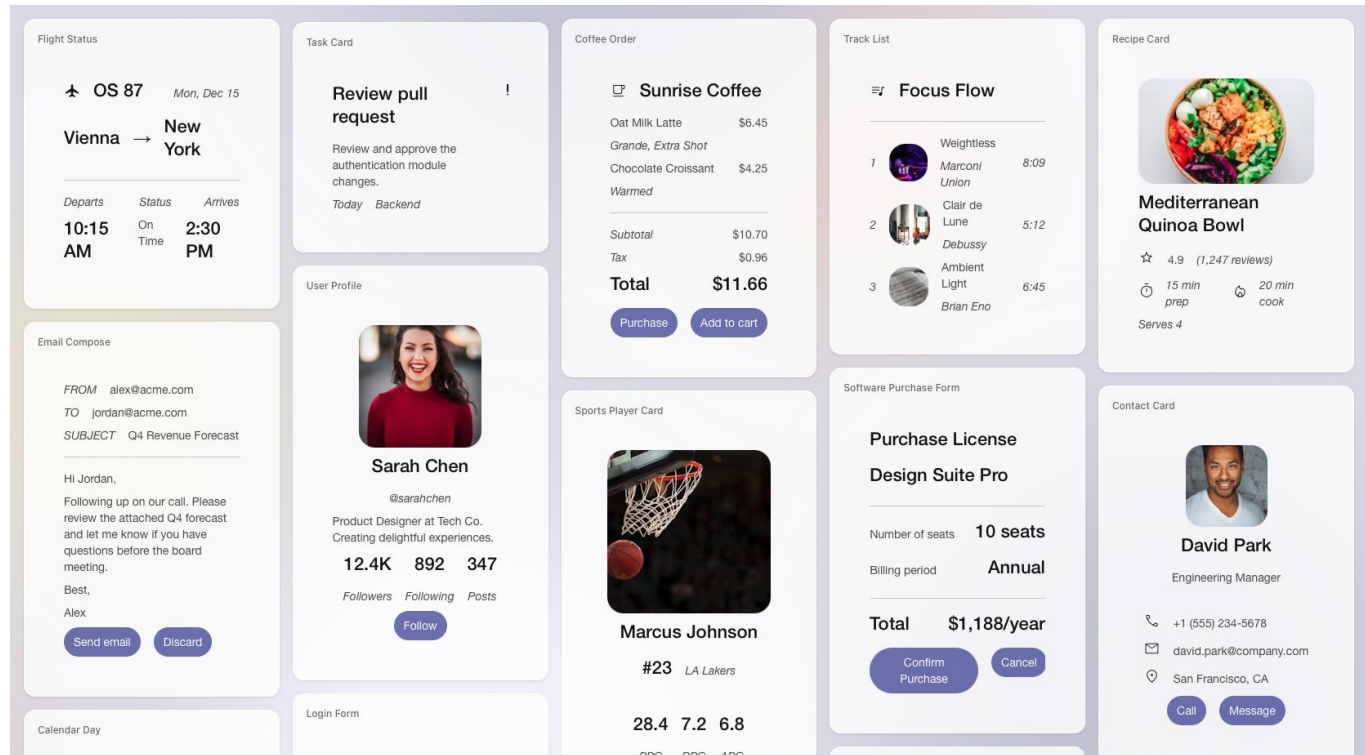
A2UI

Agent-to-UI Protocol



What is A2UI?

- A2UI is an open declarative protocol from Google for letting AI agents describe user interfaces that a client then renders with its own native widgets
- Purpose: addresses the problem of how AI agents can safely send rich UIs across trust boundaries
 - *Instead of text-only responses or risky code execution, A2UI lets agents send declarative component descriptions that clients render using their own native widgets*



Consider the Scenario...

An agent designed to book an office hours appointment with a professor (might happen in the future):

- Student: I'd like to book an appt with Prof. Irani
- Agent: OK. What day?
- Student: Tuesday
- Agent: OK. What time?
- Student: 5pm?
- Agent: He is booked at 5pm, how about 6pm, 7pm, or 9pm?

A better experience is for the agent to generate a date picker and have the student select from the list of available times.



The Core Flow

1. An Agent: which generates an A2UI response
2. A JSONL payload describing the composition of UI components and their properties
 - a. This message is transported to the client (via A2A for example)
3. The client's A2UI Renderer parses the JSONL
4. The renderer maps abstract components (e.g., type: "text") to concrete implementations in the client's codebase

The client parses each line of JSON as a distinct message and incrementally builds the UI, which is what enables progressive rendering as tokens arrive



Important to Note

The Agent that wants to generate the UI is normally not sitting within the client application and therefore, cannot edit the DOM directly. It is usually sitting on a remote server, so it can communicate, but not edit directly

So it needs to send a message of what to create/edit/delete as a stream of JSONL



Why Use JSONL?

- A regular JSON array `[{...}, {...}, {...}]` can't be safely parsed until the closing `]` arrives. With JSONL, the client parses line 1 the moment a newline arrives and starts processing it, even while line 2 is still being generated upstream.
- LLMs are also better at producing JSONL than nested JSON. Token by Token generation!
- And it's append-only friendly. The server can keep the SSE (Server-Sent-Events) connection open and append data-model updates as new lines whenever something changes, without ever rewriting earlier output



A2UI: How It Works

1 Generation

Agent (Gemini, GPT, Claude) generates an A2UI Response which is JSONL describing UI components.

2 Transport

Payload sent to client via A2A or other transport mechanism.

3 Resolution

Client's A2UI Renderer parses the JSON payload.

4 Rendering

Renderer maps abstract components (type: 'text-field') to concrete native widgets.

5 Interaction

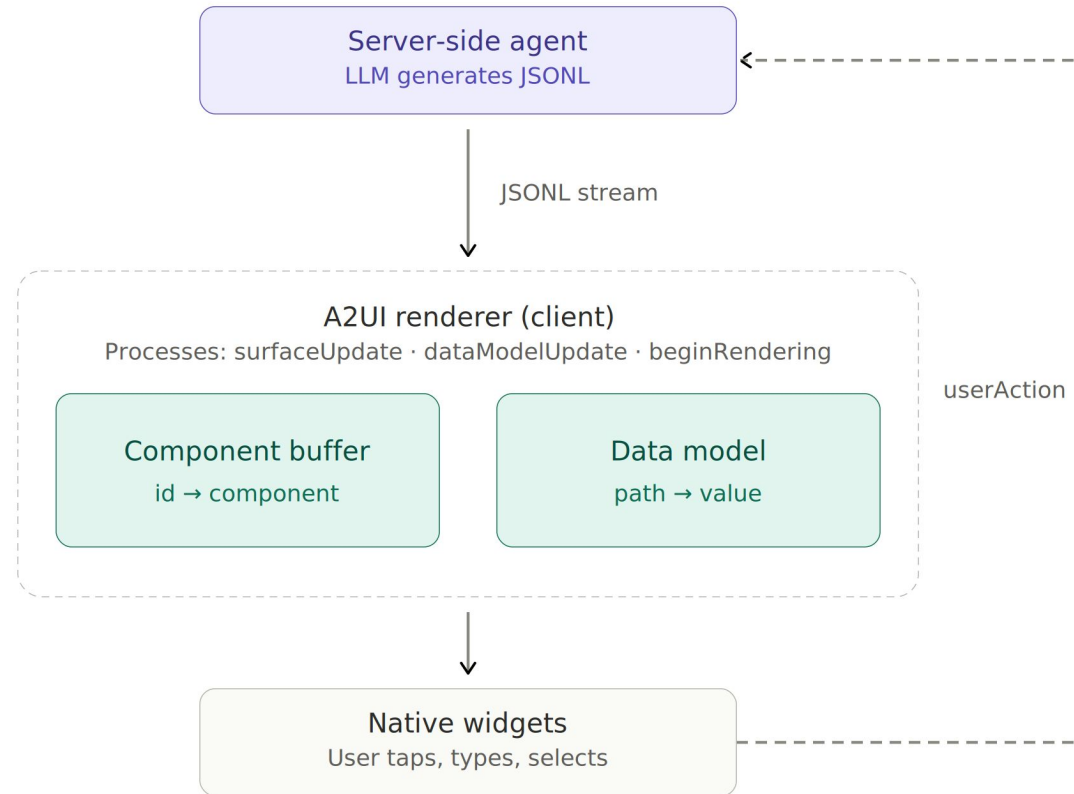
User actions (clicks, form submissions) sent back as events to the agent.

6 Update

Agent responds with new A2UI messages to update the interface incrementally.



Architecture



A2UI: The Problem It Solves

The Text-Only Problem

Most agent interactions are chat-based. When agents need structured input (dates, selections, forms), text back-and-forth is slow and error-prone.

Without A2UI (5+ turns)

Agent: 'What date?'

User: 'Dec 16'

Agent: 'What time?' → **User:** '7pm' → **Agent:** 'Guests?' → ...

With A2UI (1 interaction)

Agent generates a form with date picker, time selector, party size, and submit button. User fills everything at once.

The Security Problem

In multi-agent systems, remote agents across trust boundaries cannot directly manipulate the host app's DOM.



Sending HTML/JS via iframes is heavy and visually inconsistent



Direct DOM manipulation introduces XSS and injection risks



No standard way for agents to request structured user input



A2UI: Declarative JSON — data, not code. Client controls rendering.



A2UI: Key Design Decisions

Flat Component List

Components as flat list with ID references (not nested). LLM-friendly, supports streaming.

Framework-Agnostic

Same JSONL renders on web, mobile, desktop. One payload = all platforms.

Data Model Separation

Three layers: UI structure, application state, client rendering. JSON Pointer bindings.

JSONL Streaming

Progressive rendering. Users see UI building as the agent generates it.

Security-First

Client catalog of trusted components. Agent can only reference catalog items.

Incremental Updates

Agent updates individual components by ID without regenerating entire UI tree.



A2UI: Four Message Types

1

`createSurface`

Initialize a rendering surface with a component catalog reference.

2

`surfaceUpdate`

Describe components: flat list with IDs, types, and properties.

3

`dataModelUpdate`

Provide/update data that binds to UI components via JSON Pointers.

4

`beginRendering`

Signal the client to start rendering from a specified root component.



All A2UI Must Have Two Things

1. Component Buffer: is a flat dictionary mapping component IDs to their definitions.
2. Data Model: separate JSON object holding the UI's strings, numbers, booleans

Example: standalone.html



Important Note (1 of 2)

- Column: Vertical stacking
- Row: Horizontal stacking
- Everything will have a parent/child relationship
 - UIs are inherently trees, any non-trivial interface is "this container holds those elements, which hold these other elements"
 - A2UI specifically encodes this parent-child structure through ID references in a flat buffer rather than nested JSON, because that lets an LLM stream components in any order and lets the client buffer them incrementally without waiting for an entire nested tree to finish serializing



Important Note (2 of 2)

- The client just handles the parsing of JSON into whatever format you decide. This allows A2UI to work for any client (iOS, Android, ReactJS, etc.)
- The agent/server will create the proper format using an LLM. An example:

```
instruction = schema_manager.generate_system_prompt(  
    role_description="You are a helpful assistant...",  
    ui_description="Use cards for summaries, tables for comparisons...",  
    include_schema=True,  
    include_examples=True,  
    allowed_components=["Heading", "Text", "Card", "Button", "Table"],  
)  
agent = LlmAgent(model="gemini-flash-latest", instruction=instruction, ...)
```



A2UI: Complete Booking Form Example

Full JSONL payload — 4 messages for restaurant booking form

Message 1: createSurface

```
{"createSurface": {  
  "surfaceId": "booking",  
  "catalogId": "...basic_catalog.json"  
}}
```

Message 2: surfaceUpdate

```
{"surfaceUpdate": {  
  "components": [  
    {id: "title", component: {Text: {...}}},  
    {id: "date", component: {DateTimeInput: {...}}},  
    {id: "btn", component: {Button: {...}}}  
  ]  
}}
```

Message 3: dataModelUpdate

```
{"dataModelUpdate": {  
  "contents": [{  
    key: "booking",  
    valueMap: [  
      {key: "date", value: ""},  
      {key: "time", value: ""},  
      {key: "guests", value: 2}  
    ]  
  }]  
}}
```

Message 4: beginRendering

```
{"beginRendering": {  
  "surfaceId": "booking",  
  "root": "title"  
}}
```



A2UI: Companies & Ecosystem

Google

Created A2UI. Flutter GenUI SDK uses A2UI under the covers.

CopilotKit

Launch partner. Full AG-UI + A2UI integration. 120K+ weekly installs.

Oracle

Open Agent Spec + AG-UI + A2UI joint integration (March 2026).

AWS

AG-UI endpoint in AgentCore + FAST template with A2UI (March 2026).

Microsoft

Agent Framework AG-UI integration with A2UI rendering support.



A2UI: Key Readings

Official Site

a2ui.org specification, examples, component catalog

Launch Blog

Google Developers "Introducing A2UI" (Dec 2025)

GitHub

github.com/google/A2UI

CopilotKit Guide

"AG-UI and A2UI: Understanding the Differences"

Tutorial

CopilotKit "Build with Google's A2UI Spec" (full-stack)

The New Stack

"Agent UI Standards Multiply" (Dec 2025)



Advanced: Stream from a Server

python3 [server.py](#)

navigate to localhost:8080 in your browser to stream the JSONL from the server to the client

Production demo: <https://a2ui.southleft.com/demo>



NANDA

Networked Agents and Decentralized AI



NANDA: Intro

NANDA (Networked Agents and Decentralized AI) is an MIT Media Lab project. It is not a single protocol but an architecture for the "Internet of AI Agents"

Where A2UI, A2A, and MCP each tackle one layer (UI, agent-to-agent messaging, tool access), NANDA is trying to build the infrastructure underneath

It's the equivalent of DNS, certificate authorities, and routing for a world where trillions of agents need to find each other, prove who they are, and coordinate in milliseconds.



Core Problem

- DNS was designed for relatively static hostnames that change on the order of minutes to days
- Agents are different: the Internet is poised to host billions to trillions of autonomous AI agents that negotiate, delegate, and migrate in milliseconds, which breaks DNS-style identity and discovery
- You also need to verify what an agent claims it can do, rotate keys quickly when one is compromised, and let agents from different orgs find each other without leaking sensitive information



NANDA: What Is It?

Full Name: Networked Agents and Decentralized AI

Origin: MIT Media Lab, led by Prof. Ramesh Raskar. 18 research institutions.

Vision: *"Just as DNS revolutionized the internet, we need similar infrastructure for the Internet of Agents."*

The Four Infrastructure Gaps

1

DNS-equivalent

How do you find an agent by capability?

2

Certificate Authority

How do you verify agent identity?

3

Orchestration

How do agents coordinate at scale?

4

Attestation

How do you verify agent claims?



NANDA: Core Components

NANDA Index

Global discovery layer. Quilt-like architecture for finding agents across the internet.

AgentFacts

Schema-validated assertions: capabilities, trust scores, compliance certs, DIDs.

Dynamic Resolution

Unlike static DNS, considers availability, trust, latency, and privacy preferences.

Verifiable Claims

W3C Verifiable Credentials prevent spoofing and agent impersonation.

ZTAA

Zero Trust Agentic Access. No implicit trust between agents — every request is verified.

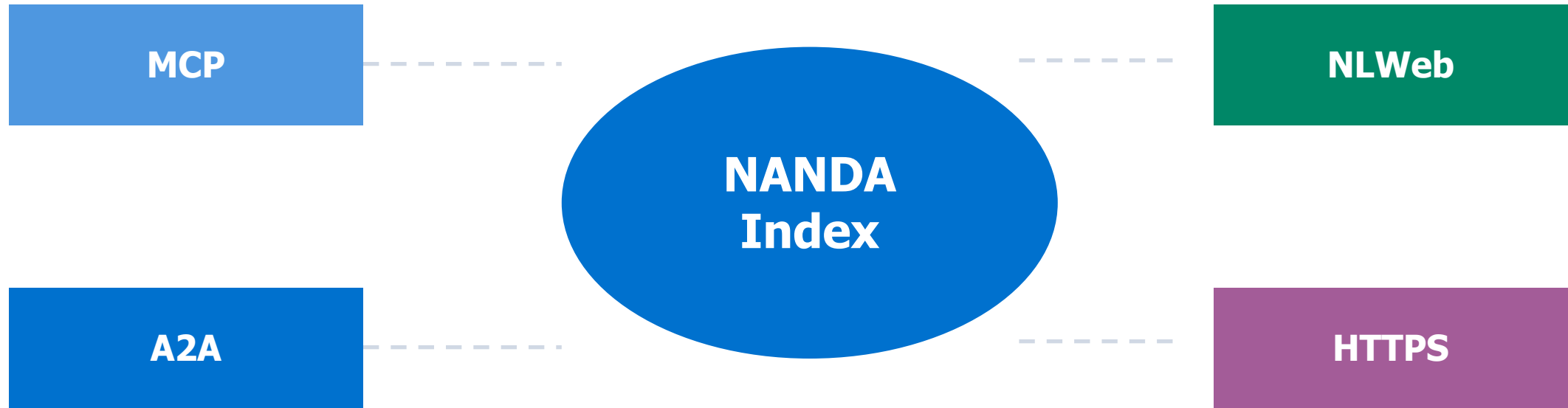
Agentic SafeSearch

Filtered queries using safety metadata to prevent malicious agent discovery.



NANDA: Cross-Protocol Interoperability

NANDA is protocol-agnostic. Agents using MCP, A2A, NLWeb, or HTTPS all register in the same global index.



Quick Start: `pip install nanda-adapter`

Examples: `nanda-pirate` | `nanda-pirate-langchain` | `nanda-sarcastic`



NANDA: Use Case for Enterprise Compliance

1

Query NANDA Index

Compliance agent queries NANDA Index for regional policy agents.

2

Receive AgentFacts

NANDA returns AgentFacts with trust scores and verified DIDs.

3

Dynamic Resolution

Dynamic resolution finds optimal endpoints based on latency and trust.

4

A2A Communication

Uses A2A protocol for structured policy requests between agents.

5

MCP Tool Connection

Each agent uses MCP to connect to legal databases and data sources.

6

Cryptographic Audit

All work is cryptographically signed and recorded for compliance.



NANDA: Readings & Discussion

Readings & Resources

MIT Media Lab: media.mit.edu/projects/mit-nanda/overview

ArXiv Paper 1: [NANDA Index architecture \(Jul 2025\)](#)

ArXiv Paper 2: [Enterprise Perspective \(Aug 2025\)](#)

Discussion Questions

- How does NANDA compare to DNS? What are the key differences for agent discovery?
- Centralized vs decentralized registries — what are the trade-offs for agent ecosystems?
- Who audits trust scores? How do you prevent gaming of the AgentFacts system?



Searching Agents

- General agent search:
 - `curl "https://hol.org/registry/api/v1/search?registries=nanda&limit=5" | jq`
- Search for a type of agent:
 - `curl "https://hol.org/registry/api/v1/search?registries=nanda&q=math&limit=5" | jq`
- Agents to support A2A:
 - `curl "https://hol.org/registry/api/v1/search?registries=nanda&protocols=a2a&limit=5" | jq`
- Extract just name and ids (ignoring everything else):
 - `curl -s "https://hol.org/registry/api/v1/search?registries=nanda&limit=10" \ | jq -r '.hits[] | "{.name} | {.uaid} | available={.available}"'`



Using Agents

- General agent search:
 - `UAID="uaid:aid:8tvwVqvJEwGqJrSai1nBAVbdkBiygpSW8bZokq8RoVMA7PaxPqVnMwGWQwECA5BS9t"; curl -s -X POST "https://hol.org/registry/api/v1/chat/session" -H "content-type: application/json" -d "{\"uaid\":\"$UAID\"}" | jq`
- Register for an account:
 - `https://hol.org/registry/dashboard`
- Set API in terminal:
 - `export REGISTRY_BROKER_API_KEY="<YOUR KEY>"`
- Use the Agent:
 - `SESSION_ID="PASTE_SESSION_ID_HERE"; curl -s -X POST "https://hol.org/registry/api/v1/chat/message" -H "content-type: application/json" -H "x-api-key: $REGISTRY_BROKER_API_KEY" -d "{\"sessionId\":\"$SESSION_ID\",\"message\":\"Hello, what can you do?\"}" | jq`



AgentKit

Build, Deploy, and Optimize Agents



AgentKit

- OpenAI builder for drag and drop editor for Agents
- Less customizations than using an SDK
- Allows for
 - Model config
 - Conditional logic
 - Pre-built tools: Search, guardrails, and even MCPS (custom)
 - etc.



AgentKit: What Is It?

A complete suite of tools for building, deploying, and optimizing agents. Announced at OpenAI DevDay, October 2025.

Pricing: Included with standard API model pricing

4M+

developers on OpenAI platform

800M+

weekly ChatGPT users

Oct 2025

DevDay launch



AgentKit: Five Components

Agent Builder

Beta

Visual canvas for creating and versioning multi-agent workflows. No-code drag-and-drop.

Connector Registry

Beta

Central admin panel for data sources. Pre-built connectors + MCP server support.

ChatKit

GA

Embeddable chat UI toolkit. Streaming, threads, tool visualization out of the box.

Guardrails

Open Source

Modular safety: PII masking, jailbreak detection, content filtering. Python + JS.

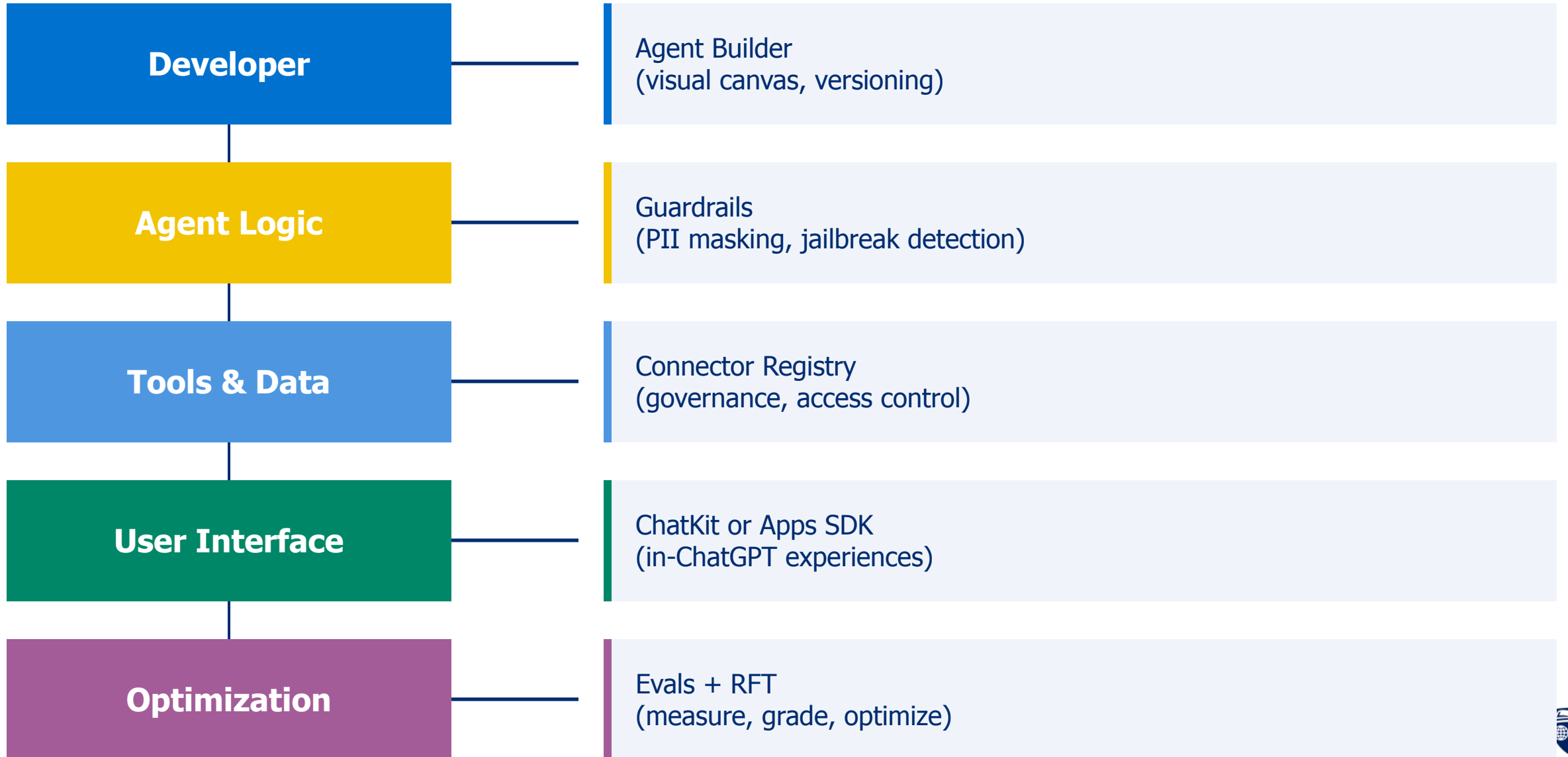
Evals

GA

Datasets, trace grading, automated prompt optimization, 3rd-party model support.



AgentKit: Architecture & Pipeline



AgentKit Apps SDK: Apps Inside ChatGPT

Open-source framework built on MCP **enabling third-party apps inside ChatGPT.**

Launch Partners

Booking.com

Canva

Figma

Spotify

Zillow

Coursera

Expedia



AgentKit: Readings & Discussion

Readings & Resources

Launch Blog: openai.com/index/introducing-agentkit/

Guardrails: github.com/openai/guardrails (open source)

Discussion Questions

- Does AgentKit create vendor lock-in to the OpenAI ecosystem? How does it compare to Google ADK?
- No-code vs code-first agent building: what are the trade-offs for enterprise adoption?
- Apps inside ChatGPT, is this the new app store, or a walled garden?



Try it Out

- We won't deploy anything, but try this at home!

<https://platform.openai.com/agent-builder/>



Browser Use

Visual Agent Interaction Paradigm



Browser Use Introduction

- An open-sourced Python library that lets AI agents control a real web browser to do tasks like:
 - Navigate sites
 - Click buttons
 - Fill forms
 - Scrape/Extract data

We used to write brittle scripts to navigate websites “click the element with this exact CSS selector”

Now we give an AI agent a goal such as “go to this site, find the pricing page, and summarize the enterprise plan”



Browser Use: Why Agents Need Browsers

Most information and services live behind web interfaces, not APIs. Agents need to navigate the web like humans do.

1 LLMs Got Good Enough

GPT, Claude, Gemini can interpret and reason about web pages.

2 Infrastructure Matured

Browserbase, Steel provide managed headless browsers at scale.

3 Economics Shifted

88% of orgs use AI regularly; 62% are experimenting with agents.

4,700%

Year-over-year increase in AI agent traffic to US retail sites
Adobe Analytics, July 2025



Browser Use: The Landscape

Open-Source Frameworks

Browser Use	78K★	Dominant. 89% WebVoyager benchmark.
Crawl4AI	51K★	LLM-friendly web crawler → markdown.
Stagehand	21K★	Self-healing actions. Browserbase.
Skyvern	20K★	Vision-LLM. Screenshot-based. YC-backed.

Consumer AI Browsers

Perplexity Comet	Jul 2025
ChatGPT Atlas	Oct 2025
Chrome Auto Browse	Jan 2026
Opera Neon	Sep 2025

Infrastructure

Browserbase: \$40M Series B, \$300M valuation, 50M sessions. Managed headless browser infrastructure.

Bright Data: 1M+ concurrent browser sessions. Enterprise web data platform.



Browser Use: Market Data & Legal

4,700%

YoY increase in AI agent traffic to retail

Adobe Analytics

38%

of consumers used AI for shopping by Q3 2025

Industry surveys

\$24.25B

browser automation market 2026

Market research

\$84.22B

projected market by 2034

CAGR growth forecast

Legal Frontier

Amazon v. Perplexity (Jan 2026): First major lawsuit against agentic browser technology. Core issue: agents not identifying themselves in User-Agent headers. Sets precedent for autonomous web browsing rights.



Browser Use: Architecture Patterns

DOM-Based (Browser Use)



Vision-Based (Skyvern, CUA)



Key Difference: DOM-based agents read the page structure directly (faster, cheaper). Vision-based agents see screenshots like humans (more robust to anti-bot measures, works on any page).



DOM-Based vs Vision-Based

Dimension	DOM-Based	Vision-Based
How it works	Parses HTML DOM tree, selects elements	Takes screenshots, vision-LLM identifies targets
Speed	Fast: direct element access	Slower: screenshot + inference per step
Cost	Lower: text tokens only	Higher: image tokens per step
Robustness	Breaks on dynamic/JS-heavy pages	Works on any visual page layout
Anti-Detection	Easier to detect (DOM inspection)	Harder to detect (mimics human)
Best For	Structured sites, forms, data extraction	Complex UIs, anti-bot sites, visual tasks
Examples	Browser Use, Crawl4AI, Stagehand	Skyvern, CUA, Claude Computer Use



Browser Use: Python Example

```
from browser_use import Agent
from langchain_openai import ChatOpenAI

agent = Agent(
    task='Go to Hacker News, find the top 5 stories, '
        'and return their titles and URLs.',
    llm=ChatOpenAI(model='gpt-4o-mini')
)

result = await agent.run()

# Agent autonomously:
# 1. Navigates to news.ycombinator.com
# 2. Reads the DOM tree
# 3. Extracts top 5 story titles + links
# 4. Returns structured result
```

Import

Browser Use + LangChain integration.
Supports OpenAI, Anthropic, local models.

Task Definition

Natural language task description. Agent plans and executes autonomously.

Execution

Single async call. Agent handles navigation, DOM parsing, and extraction.

Autonomous Steps

No explicit selectors or URLs needed. LLM decides what to do.



Browser Use: Security Concerns

Critical

Prompt Injection

Hidden text (display:none) on web pages can manipulate agent behavior. Agents read content humans cannot see.

High

Cross-Domain Leakage

Multi-tab agents may inadvertently leak data between sites. Session cookies and auth tokens at risk.

High

Trust Boundary

Distinguishing user instructions from website content is fundamentally hard for LLM-based agents.

Medium

Bot Detection

Should agents identify themselves? CAPTCHAs? Sites increasingly blocking automated access.



Browser Use Example

Installation (must be on Python 3.12 or higher):

```
python3.12 -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install "browser-use>=0.12" anthropic playwright
```

```
playwright install chromium
```

```
export ANTHROPIC_API_KEY=sk-ant-...
```

```
python browser_use_demo.py
```



Browser Use Example (Windows Powershell)

Installation (must be on Python 3.12 or higher):

```
py -3.14 -m venv .venv
```

```
.venv\Scripts\Activate.ps1
```

```
pip install "browser-use>=0.12" anthropic playwright
```

```
playwright install chromium
```

```
$env:ANTHROPIC_API_KEY="sk-ant-..."
```

```
python browser_use_demo.py
```

**** I am unable to test and debug this since I do not have a Windows machine**



Browser Use: Readings & Discussion

Readings & Resources

GitHub: github.com/browser-use/browser-use (78K stars)

Browserbase: browserbase.com: managed browser infrastructure

Skyvern: github.com/Skyvern-AI/skyvern (vision-based)

Discussion Questions

- Should websites have the right to block AI agents? What about robots.txt for agents?
- CAPTCHA ethics — are they still effective? Should agents identify themselves?
- 4,700% traffic growth — what are the implications for web infrastructure and costs?
- Vision-based vs DOM-based: which approach wins long-term?



Agents.md

Repo-level Agent Guidance



AGENTS.md: What Is It?

A simple, open format for guiding coding agents. *Think of it as a README for agents.*
Problem: Senior engineers carry unwritten rules. AI coding agents don't have this tribal knowledge.

Origin

OpenAI Codex, Amp, Google Jules, Cursor, Factory

Governance

Donated to AAIF (Linux Foundation), December 2025

Format

Standard Markdown — no special syntax needed

Fun fact: OpenAI's own repo has 88 AGENTS.md files

Adopted By

OpenAI Codex

Amp

Google Jules

Cursor

Factory



AGENTS.md: Discovery & Precedence

1

Global

~/.codex/AGENTS.override.md → ~/.codex/AGENTS.md

2

Project Walk

Walk from root to current dir. At each level: .override.md → .md → fallbacks

3

Merge

Concatenated root-down. Closer = later = higher priority.

4

Size Limit

32 KiB default. Split across nested dirs if needed.

Recommended Sections

Build & Test

Architecture

Security

Git Workflows

Conventions

Priority Rule: Files closer to the current working directory take precedence. Override files (.override.md) always take priority over regular files at the same level.



AGENTS.md: Example File

AGENTS.md

```
# Build & Test
- Run `pytest tests/ -v` after any change
- Integration tests: `make test-integration`
- Lint: `ruff check . && mypy src/`

# Architecture
- FastAPI service in `src/api/`
- Domain logic in `src/core/` – never import from api layer
- All DB access through `src/repositories/`

# Security
- No secrets in code – use `VAULT_ADDR` env var
- All endpoints require JWT auth (except /health)
- Sanitize user input before DB queries

# Git Workflows
- Branch naming: `feat/`, `fix/`, `chore/`
- Squash merge to main; keep PR title as commit msg
- Tag releases: `v{major}.{minor}.{patch}`
```

```
# Conventions
- Type hints required on all public functions
```



AGENTS.md: Override Patterns

```
project-root/
├── AGENTS.md                ← shared conventions
├── services/
│   ├── payments/
│   │   └── AGENTS.override.md ← payments team rules
│   ├── auth/
│   │   └── AGENTS.override.md ← auth team rules
│   └── billing/
│       └── AGENTS.md          ← billing-specific
└── guidance
    └── libs/
        └── shared/
            └── AGENTS.md      ← shared library
```

Root Level

Shared conventions apply to all agents across the entire project.

Override Files

Team-specific rules ADD to root guidance. They don't replace it.

Precedence

Closer files appear later in prompt and take precedence over distant ones.

Key Insight: Overrides ADD to root guidance — they don't replace it. Think of it as layered configuration, similar to CSS cascading.



AGENTS.md: Best Practices

Be Specific

"Run pytest tests/ -v" not "run the tests"

Require Proof

Tests, lint, type check. "Don't mark complete until verification passes."

Document Gotchas

Domain vocabulary, common pitfalls, environment quirks.

Treat Like Code

Pull requests, reviews, CODEOWNERS for AGENTS.md files.

Start Small

One runbook per agent. Lock down permissions. Add complexity incrementally.

Brevity Wins

Keep under 32 KiB. If it's too long, split across nested directories.



AGENTS.md: Real-World Patterns from OpenAI

88 AGENTS.md files in the OpenAI Codex repository

Mandatory Skills

`$code-change-verification` — tests, lint, type check. Can't mark complete until it passes.

API Compatibility

"Preserve positional meaning of constructor parameters. Append new optional ones at end."

Environment Awareness

"CODEX_SANDBOX_NETWORK_DISABLED=1 set. Skip network-dependent tests."

Scoped Tests

"Run cargo test -p codex-tui first. Full suite only if shared crates changed."

Changeset Validation

Agent must create/update changesets AND validate bump levels match diff.



AGENTS.md vs Alternatives

Feature	AGENTS.md	.cursorrules	copilot-instructions.md	CLAUDE.md	System Prompts
Vendor Lock-in	None	Cursor only	GitHub only	Anthropic only	Platform-specific
Hierarchical	✓ Full	✗	✗	✓ Partial	✗
Override Support	✓	✗	✗	✗	N/A
Version Controlled	✓	✓	✓	✓	Usually not
Governance	AAIF (Linux Foundation)	Cursor Inc.	GitHub/Microsoft	Anthropic	Varies
Multi-Agent	✓ Designed for	Single agent	Single agent	Single agent	Varies
Discovery	File-system walk	Root only	Fixed path	Root + nested	API config
Size Limit	32 KiB default	None specified	None specified	None specified	Token limit



AGENTS.md: Readings & Discussion

Readings & Resources

Official Specification:

agents.md — the canonical format specification

Documentation:

OpenAI Codex docs — discovery & precedence rules

Case Study:

"Using skills to accelerate OSS maintenance" — real-world adoption patterns

Discussion Questions

- 1 Should AGENTS.md be as standard as README.md in every repository?
- 2 Could AGENTS.md be weaponized — malicious instructions in open-source repos that trick agents?
- 3 How will AGENTS.md evolve as agents become more capable and need less explicit guidance?



Looking Ahead

Repo-level Agent Guidance

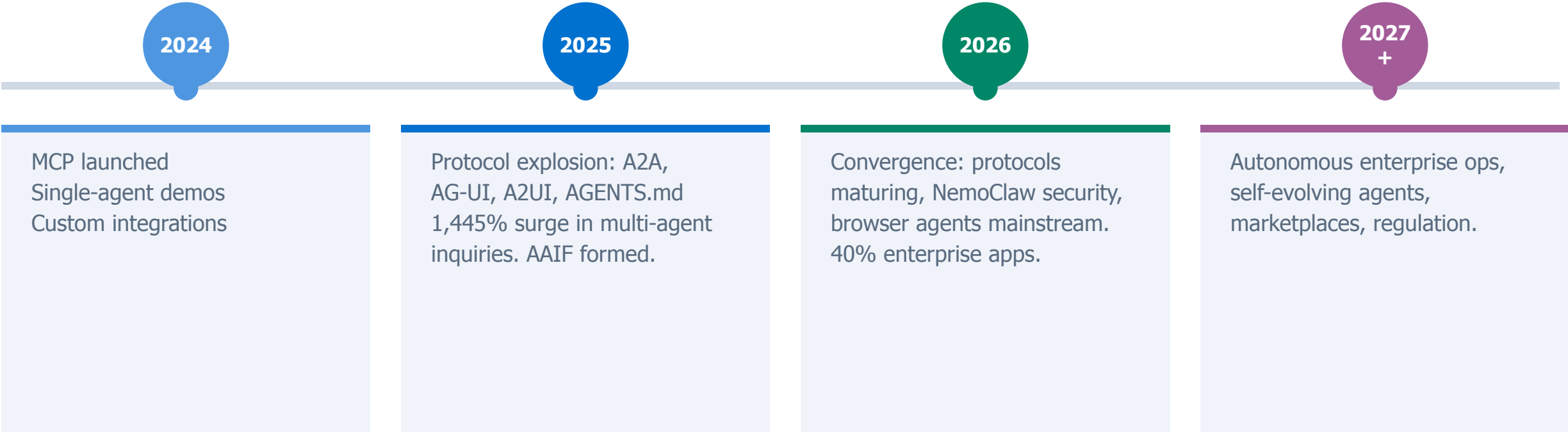


The Complete Protocol Stack

Layer	Protocol	Purpose	Status
Discovery	NANDA	Find & register agents across networks	Emerging
Agent ↔ Agent	A2A	Cross-vendor agent collaboration	Active — Google-led
Agent ↔ Tool	MCP	Standardized tool/resource access	Mature — Anthropic-led
Agent ↔ UI	A2UI + AG-UI	Rich, real-time human interfaces	New — Google + CopilotKit
Agent Guidance	AGENTS.md	Repo-level instructions for coding agents	Growing — AAIF/Linux Fdn
Runtime Security	NemoClaw	Enterprise guardrails & sandboxing	Enterprise — NVIDIA
Web Access	Browser Use	Autonomous web navigation & interaction	Mainstream — open-source



Industry Trajectory



 **Warning:** 40%+ of agentic AI projects will be canceled by 2027 (Gartner) — costs, unclear value, and insufficient risk controls remain the top barriers to production deployment.



Key Takeaways

1 The protocol stack is: MCP + A2A + A2UI/AG-UI + NANDA = coherent, complementary architecture.

2 A2UI $\neq$ AG-UI: Blueprint (what) vs plumbing (how). They work together.

3 Discovery is the missing layer: NANDA = DNS for agents.

4 Security cannot be an afterthought: 2,000 unauth MCP servers, prompt injection, impersonation.

5 Enterprise $\neq$ demo readiness: NemoClaw exists because OpenClaw works for demos, not production.

6 AGENTS.md is deceptively simple: 88 files in OpenAI's repo show real-world complexity.

7 Browser agents going mainstream: 4,700% traffic growth, legal battles, consumer products.



Further Reading: Master List

Essential Reading

- Google "Developer's Guide to AI Agent Protocols" (Mar 2026)
- StackOne "120+ Agentic AI Tools" — landscape survey
- A2A launch blog — Google's agent-to-agent vision
- A2UI launch blog — Google's agent-to-UI framework
- NemoClaw TechCrunch coverage — enterprise angle

Deep Dives

- NANDA ArXiv papers — discovery protocol formalization
- AgentKit launch blog — modular agent construction
- Browser automation landscape guide — full survey
- AGENTS.md + OpenAI skills blog — real patterns
- AG-UI CopilotKit docs — streaming UI protocol



Tools to Explore

A2A

```
pip install a2a-python
```

A2UI

```
github.com/google/A2UI
```

NANDA

```
pip install nanda-adapter
```

Browser Use

```
pip install browser-use
```

NemoClaw

```
github.com/NVIDIA/NemoClaw
```

AGENTS.md

```
agents.md
```

Frameworks

Google ADK

LangGraph

OpenAI Agents SDK

CopilotKit



Open Questions for the Field

Governance

Who controls agent registries? What happens when agents make mistakes?

Economics

How are agents priced in multi-agent workflows? Who pays $A \rightarrow B \rightarrow C$?

Security

2,000 unauth MCP servers. How do we secure autonomous agents at scale?

Standards

Will MCP + A2A + A2UI + AG-UI + NANDA converge or fragment?

Legal

Amazon v. Perplexity — is autonomous browsing fair use?

